

Relatório — Fase 1 (concluída e validada): Contrato de saída (Inserção A)

Projeto: LangNet · caso-teste ClinIA · **Data:** 2026-08-13 · **Executor:** Claude **Commit selo:** c0c4b84 · **Checkpoint de rollback (antes da fase):** a69a0c3

A Fase 1 introduz o **contrato de saída (JSON Schema)** por **task agêntica** e a **validação/reparo da saída do agente no ws-server**. É a primeira fase a mexer no **runtime do gerador**. **Não altera telas** — a mudança é no ws-server + no `tasks.yaml` gerado. Abaixo: o que foi feito e as **provas reais**.

1. O que foi implementado

No gerador (`backend/agents/langnetagents.py`): - `_derive_output_schema(task_name, task_cfg, model)` (+ `_parse_expected_output_fields, _nonnull_cols, _enum_options`): deriva um **JSON Schema de saída** por task agêntica, do `expected_output` **cruzado** com as colunas `NOT NULL` /tipo da entidade que melhor casa. **required só para colunas NOT NULL** (mínimo — não sobre-especifica, lição do SWE-bench Verified). `nivel_confianca` (FLOAT) vira `number` com `coerce_from_enum`. - `_annotate_tasks_output_schema`: injeta `output_schema`: por task no `tasks.yaml` gerado. - `_coerce_to_schema(raw, schema)` (helper nos adapters): **desembrulha** `{raw}` /string → objeto, **coage** por tipo (via `_cv: enum→float, dict→JSON`) e **valida** os `required`. Retorna `(obj, faltantes)`. - **Bloco no template do ws-server** (`_execute_task`): valida a saída do agente contra o `output_schema`; faltando obrigatório → **1 retry** com o schema reforçado no prompt; persistindo → **erro explícito (fail-loud)**, sem emitir `task_completed` com saída incompleta.

Harness (`tools/regen/`): `regen_wsserver.py` (injeta `output_schema` + regenera `websocket_server.py`) e `fault_inject.py` (teste de fail-loud); `regen.sh` ganhou o alvo `wsserver`.

2. Provas de validação (saídas reais)

2.1 Derivação do schema (task-chave `pre_atendimento_cardiologia`)

```
{ "type":"object", "required":["hipoteses","nivel_confianca"],
  "properties":{"hipoteses":{"type":"string"},
    "nivel_confianca":{"type":"number","coerce_from_enum":{"...","alta":0.9,"média":0.7,"baixa":0.4}},
    "exames_sugeridos":{"type":"string"} } }
```

✓ required **mínimo** (só as colunas `NOT NULL`); `nivel_confianca` como `number`.

2.2 Unit do `_coerce_to_schema` (4 casos)

Caso	Entrada	Resultado
Envelope {raw:...} + enum + dict	<code>{"raw":"{..., nivel_confianca:'alta', hipoteses:{...}}"</code>	<code>nivel_confianca=0.9</code> (float), <code>hipoteses</code> → JSON string , <code>faltantes []</code>
Cerca markdown	<code>```json{...}```</code>	parseado
	<code>{"nivel_confianca":"média"}</code>	faltantes ["hipoteses"]

Caso	Entrada	Resultado
Incompleto (falta hipóteses)		
Opcional ausente (exames).	<code>{"hipoteses":"x","nivel_confianca":"alta"}</code>	faltantes [] (não reprova)

2.3 E2E (./smoke.sh) — regressão zero + saída limpa

```
[1..4] ok=true (cadeia acumulada até prontuario_id) [5] ok=true
[verify] atendimento=True | encaminhamento(medico+esp)=True | prontuario(liga pre_diag+enc)=True
[smoke] VERDE
```

E o `pre_diagnostico` persistido pela UI, **com a saída já sob contrato**:

```
nivel_confianca = 0.7 (numérico — coagido do enum do agente)
hipoteses = {"angina_peito":0.85, "infarto_agudo_mi...} (objeto → JSON string)
```

✓ O frontend recebeu **objeto limpo** (sem `{raw}`) e o valor chegou **numérico** — sem os remendos.

2.4 Fault-injection (fail-loud) — fault_inject.py

Adicionei um campo obrigatório impossível (`campo_inexistente_teste`) ao `output_schema` da task e chamei-a:

```
RESULTADO: error (fail-loud OK) :: saída do agente não cumpre o contrato de saída;
faltam: campo_inexistente_teste | faltantes: ['campo_inexistente_teste']
```

✓ O ws-server respondeu **error** (não `task_completed`). Logo, no fluxo real, o `runTask` do frontend **rejeita** e a persistência **não ocorre** com saída incompleta. (*tasks.yaml restaurado após o teste.*)

3. Telas

A Fase 1 não altera nenhuma tela. A mudança é no ws-server (validação da saída) e no `tasks.yaml` (schema). A prova visual do fluxo pela UI já consta no relatório da Fase 0 (o smoke desta fase seguiu **VERDE**, exercitando as mesmas telas).

4. Benefício e trilha de commits

- **Benefício:** elimina **na fonte** a família de bugs de saída (`{raw}`, `enum↔float`, campo faltando → `NOT NULL` silencioso). **Aposenta** `parseAgentResult / _cv` como remendos. Combate ao *specification gaming* ("respondeu algo ≠ respondeu o contrato").
- **Rollback:** `a69a0c3` (ANTES da Fase 1). **Selo:** `c0c4b84` (Fase 1 concluída e validada).

5. Próximo passo

Fase 2 — bundle OKF de contexto (Inserção E): emitir o domínio como conhecimento OKF e ligar os agentes a ele como contexto aterrado (ataca a alucinação na raiz). Será precedida pelo commit **CHECKPOINT — ANTES da Fase 2**, descrevendo o que a fase implementa.